

Zeal: PIR for Non-Cooperative Databases

Javin Zipkin
UCSB

Ofir Dvir
UCSB

Divyakant Agrawal
UCSB

Trinabh Gupta
UCSB

Soamar Homsi
AFRL

Abstract

Private Information Retrieval (PIR), a set of techniques from the literature on cryptography, enables the retrieval of data from a public database while concealing the intent of those querying it, even when the database itself is untrusted. While the scalability of PIR has improved in recent years, its applicability remains limited due to the assumption that databases cooperate with users. PIR schemes usually require the database or website administrators to perform costly operations beyond servicing requests, which they have little incentive to do. In this paper, we introduce a new direction of PIR research that eliminates requirement of any special cooperation from the database and assumes the presence of a strong adversary that controls not only the database but also any third parties involved in the system. We present Zeal, the first ‘non-cooperative’ PIR scheme that defends against a strong adversary. We also implement Zeal on AWS and evaluate its performance. Zeal has roughly three to four minutes of latency for a database with one million records, improving upon the latency of a naive solution by a factor of 50. We also prove and quantify Zeal’s security using a differential privacy guarantee. ¹

1 Introduction

In the PIR problem, a user retrieves an item from an untrusted public database without revealing which record they are specifically interested in [13]. This problem, which at first appears paradoxical, has been extensively researched for the last three decades, and recent schemes can execute PIR very efficiently even for large databases and large amounts of clients [2, 3, 5, 23, 33, 36]. PIR can be applied to browsing public databases or can be used as a subroutine for secure search engines [22] or browser extensions [15].

Unfortunately, existing research on PIR makes assumptions about public databases that are often unrealistic in the

real world. First and foremost, existing schemes require the database server to perform computations beyond servicing requests that database operators must implement beforehand. This might include masking [44], representing the data as a matrix and multiplying it by a vector [32], or cryptographic operations [11, 22]. Database operators have no incentive to cooperate, especially considering that the computation overhead is often large. Many schemes further assume the database is copied among multiple heterogeneously trusted servers [43] and rely on the user contacting all of them, but such copies may be inaccessible, inconsistent, or nonexistent. Furthermore, since Beimel et al. [7] proposed PIR schemes with preprocessing, the most efficient schemes require an offline phase where clients compute on query-independent data, but this relies on the database being static. Preprocessing for nonstatic databases exists [30] but it still requires database cooperation.

The question we proceed to answer in this paper is: Can we perform PIR with no special cooperation from the relevant database (only querying for specific plaintext items) while still being able to hide users’ true intentions from a strong adversary?

The novel assumptions of this work present an enormous challenge. The only baseline solution that provides a strong security guarantee while following them is the naive solution to PIR. This trivial solution consists of a simple download of the entire database, as a user’s specific intent cannot be deduced when they have requested every item. However, this is prohibitively expensive for users and is far too slow for large databases. At the same time, sending queries to the database is the only way to interact with the database since it is minimally cooperative. In a non-cooperative setting, downloading every item in the database is the intuitive lower bound for PIR, as if we do not query a given item in the database, the database learns the user is not interested in it. To work within this context, we cannot use existing PIR techniques and latency will necessarily be higher than in the cooperative setting, but we can amortize this lower-bounded cost among multiple queries and offload the majority of the work onto third-party

¹Approved for Public Release on 06 Feb 2026. Distribution Is Unlimited; Case Number: AFRL-2026-0557

infrastructure rather than users.

This paper presents Zeal, a PIR scheme that provides strong provable security in the case of an uncooperative database and untrusted third parties. To address non-cooperation, Zeal creates fake or ‘dummy’ database queries to confuse adversaries and lets many users simultaneously share the benefits of this chaos. To address adversaries that may have extensive side information about users [34], Zeal uses a differential privacy guarantee that specifies and minimizes the chance of leakage. To take advantage of untrusted third parties, Zeal routes requests through a mixnet-inspired anonymity system [39] where a chain of untrusted proxy servers compute on queries obliviously and randomly.

With Zeal, databases do not have to use any special protocols or modify their contents and there is no offline preprocessing phase that relies on a static database. Crucially, Zeal offers $O(1)$ client-side CPU, network overhead, and memory usage to clients. Zeal also amortizes its server-side costs among dishonest and/or honest users.

In this paper, we make the following contributions:

- We introduce a new variant of PIR research assuming uncooperative database operators, a strong adversary that observes any third parties, and no preprocessing.
- We design and implement Zeal, a novel solution for the PIR problem. Furthermore, we provide a formal security proof that our solution is rigorously secure when all but one proxy server and all non-target users are dishonest.
- We optimize Zeal and run experiments to test its efficiency. Our results show that Zeal outperforms the naive solution in latency by about 50x and client communication costs by at least 10^6 x when databases have millions of records.

2 Problem Description

This paper is focused on the private information retrieval (PIR) problem. In this problem, a target user must retrieve the i th element from an integer-indexed database, called DB , without revealing the value of i to the database or its operators. We must ensure both correctness and security: the user must receive their intended record while hiding i .

As discussed in the introduction (§1), we wish our solution to be widely applicable. This requirement introduces additional constraints, giving the problem a novel flavor relative to prior PIR work within the literature on cryptography [3, 28, 37].

No database modification. We assume that the database DB that the user queries is not cooperative with the users. It services queries responsively, but performs no special computations, such as homomorphic encryption operations [41] or reformatting of the database. Our motivation for this constraint is that in real life there is no reason to expect databases

to follow a given protocol. This is especially true if we wish our solution to be used widely on the Internet.

No database copies. We assume that there is only one master copy of the data source. This constraint implies that we cannot base our solution on assumptions of synchronized non-colluding copies of the server, as is common in traditional PIR. Again, this constraint is motivated by the desire for our solution to be usable widely across the Internet.

No database preprocessing. We assume that DB is mutable and can be updated at any time. The database operator does not maintain database versions. This constraint implies that our scheme cannot assume that the DB is static, and can be preprocessed. A user may still perform computations before querying DB , but these steps must not rely on the specific contents of DB .

One solution that meets all of these constraints is that of trivial PIR. In this solution, a user downloads values at all indices in the DB . Naturally, this solution is quite expensive. Thus, our goal is also to minimize overheads. Ideally, a user’s CPU and network communication overheads should be within a constant factor of the *non-private* baseline (a simple query) no matter the size of the database. It would be unrealistic to require constant overheads for the database, but they should still be lower than the naive solution.

2.1 Threat model

We assume a strong semi-honest (honest-but-curious) adversary who can compromise the entire infrastructure. All parties other than the target user, including additional users simultaneously querying the database and any intermediary servers, are assumed to be semi-honest and working in coordination with the database.

Specifically, the adversary has these abilities:

- The adversary controls the database. It sees the query and database contents and the identity associated with all queries.
- All non-target users are semi-honest adversaries. The adversary may force these users to query for specific indices. These users reveal all of their privately scoped variables and cryptographic keys to the adversary.
- All involved infrastructure that may be needed, e.g., third-party servers interacting with the database and/or users, also reveal their variables and secrets. We use multiple proxy servers and assume one of these is honest, but the target user does not know which server this is, and neither do we as the engineers designing the system. This is the *anytrust* threat model [40]. To be clear, the assumption of non-collusion between instances of third party infrastructure is a weaker assumption than that of multi-server PIR. In multi-server PIR, the database operator must maintain heterogeneously trusted consistent

replicas of the database. In our construction, we can set up third-party proxies to apply to any database, and such proxies do not have to know the database contents or apply any synchronization techniques. This assumption is realistic in settings where the PIR users access cloud servers hosted by different entities in different locations. For example, the PIR users could rely on some servers in AWS and others in DigitalOcean. To break non-collusion assumptions, an adversary would then have to compromise the security of many different cloud servers. Thus, deployment is flexible.

- All adversarial parties (i.e. the database, other users, and dishonest intermediate servers) collude and share their information.
- The adversary is assumed to have already built a strong profile about the target user’s specific interests. This means that they already know the target user is only interested in some small subset of database records. We want to reckon with a strong adversary, so we say that this set is a pair of only two indices (i, j) . Thus, if the adversary can simply determine that the target user is *not* interested in the record numbered i , they may use simple process-of-elimination to determine that the user’s intent is j , and vice versa. Furthermore, the target user does not know the specific pair (i, j) that the adversary has determined. This is challenging: Any dummy requests that refer to indices other than this pair are unhelpful, since the adversary already knows the user is not interested in such records.

However, the adversary is not malicious. Network traffic may not be blocked or manipulated. All intermediate servers, dishonest users, and the database itself follow protocols correctly and responsively.

Non goals. We need not conceal the user’s identity nor the fact that they are querying *DB*, but their specific intent must be hidden.

3 Related Work

PIR with server cooperation and preprocessing. Most existing PIR schemes rely on the database operators cooperating with users. In these schemes, the database must be set up specifically to provide a PIR protocol, as it may not send plaintext records to users at all [13, 27, 29].

Modern PIR schemes that rely on server cooperation also usually rely on preprocessing. In PIR with preprocessing, users must download and compute on some query-independent data before interacting with the database. This means that any data used during preprocessing must be entirely independent of the user’s desired record [2, 23, 32, 44]. Preprocessing is an efficient strategy for PIR, but Zeal aims not to use preprocessing in order to apply to mutable databases

that would require expensive preprocessing steps to be redone constantly. Ma et al. [30] address this issue via preprocessing data that can be adjusted incrementally, but their scheme (like all schemes with preprocessing) still requires database cooperation.

PIR with no cooperation. Toledo et al. [37] introduce weakly-private PIR schemes that do not require database cooperation. Similarly to Zeal, they rely on randomly generated cover traffic that is shared between users to obscure intent. Most of their schemes only function when the database is copied over multiple servers and users may access all of them, but they also prove that a single-server scheme consisting of anonymized dummy requests is weakly private. While similar to Zeal, this scheme requires users to generate dummies themselves and assumes that compromised users do not reveal any secrets they generate, meaning that their scheme is both less efficient and uses a weaker threat model compared to Zeal. Specifically, their scheme relies on a large number of trusted users, while our scheme assumes that all but one user could be compromised by the adversary and that their actions could be observed or artificially controlled.

GooPIR [15] is similar to Zeal in that it relies on the creation of dummy queries to obscure the user’s intent. With GooPIR, the user references statistics about the relative popularity of database records to pseudorandomly generate a set of k dummy queries with similar popularity. The resulting security guarantee is called $h(k)$ privacy, since the adversary still learns some information about the user - namely, that they are interested in one of k possible items. While GooPIR is an efficient scheme, its strategy is vulnerable to attacks: Mokhtar et al. and others [6, 31] note that adversaries can observe a sequence of queries from the target user to mount reidentification attacks [34] by identifying a common topic among query sets and eliminating dummy requests that appear unrelated. Thus, GooPIR’s reliability depends on the level of side information an adversary has about a targeted user. There are two key novelties that separate Zeal from GooPIR: First, Zeal’s threat model assumes the adversary has already built a detailed profile about the user, which accounts for the attacks leveraged against GooPIR. Second, Zeal does not require the presence of statistics about the popularity of records in the database.

Mixnets. A mixnet is an anonymity system that relies on a network of intermediary servers to group and shuffle online communications from multiple users [10]. If any one intermediary server in the mixnet is honest, all users are provided anonymity with respect to one another.

Vuvuzela, a protocol that provides private peer-to-peer text messaging rather than PIR, uses a similar anonymity system that generates cover traffic in addition to mixing its input [39]. This technique is superficially quite similar to Zeal, which also mixes and adds cover traffic, but Vuvuzela does not solve the PIR problem we define (§2) because it assumes the content of the queries - in their case, the actual text messages being

Solution	No DB Cooperation	No Preprocessing	No DB Copies	Strong Threat Model
Toledo et al. [37]	✓	✓	X	X
SimplePIR [23]	X	X	✓	✓
Incremental PIR [30]	X	X	✓	✓
GooPIR [15]	✓	✓	✓	X
Zeal	✓	✓	✓	✓

Figure 1: Comparing the features of some prior PIR solutions

sent - is already hidden. Vuvuzela conceals the identity and active/inactive status of users, while Zeal conceals their intent.

Tor [14] is a popular anonymity system that uses mixnet principles to provide low-latency anonymous communication on the Internet. This may help users browse untrusted sources of information privately, but again, Tor conceals identity rather than intent. Tor also has no rigorous guarantee of security and de-anonymization attacks against Tor are prevalent [26].

ORAM. *Privately owned* data stores are subject to attacks even when their contents are fully encrypted. Adversaries can use metadata such as the timing, frequency, and sequence of memory accesses made to encrypted data stores to gain sensitive information about them. An Oblivious RAM or ORAM [19] defends against adversaries who could abuse this metadata by making the same memory accesses for all inputs. For example, a private database that uses ORAM may appear to obviously make the same memory accesses no matter which record a user has queried. A recent approach to this problem is Pancake [21]. Similarly to Zeal, Pancake conceals the intentions of users with random dummy queries and query batching to make all queries appear roughly the same, no matter their contents. However, ORAM solutions do not solve the PIR problem because they assume the content of the database and queries to it have already been protected with an encryption key the adversary does not have.

Differential privacy techniques. Zeal’s weakly private security guarantee is defined using (ϵ, δ) differential privacy. A small number of PIR schemes, including the aforementioned schemes by Toledo et al., use differential privacy, [4, 37] but there are many models of differential privacy that can be used for a wide range of applications. In classical differential privacy, a trusted data provider answers queries from users while adding noise intentionally so that clients do not know the exact answers to their queries and thus do not learn sensitive information about the database [9, 16]. This can be applied to many settings, such as privacy-preserving data analytics [17, 42]. The batched differentially private scheme by Altab et al. [4] uses similar techniques to Zeal, but they also require database cooperation and their scheme is only applicable when the number of real queries is close to or greater than N . Our work aims for a wider applicability.

Another variant of DP is Local Differential Privacy (LDP), where the data provider is untrusted and noise must be locally added to client input rather than to the database itself [1, 20]. This setting is closer to that of Zeal, as we add noise to query inputs rather than to responses, but in Zeal, users do not add

any noise locally. LDP can be used to implement randomized response techniques (RRT), where inputs to a dataset may be deleted as well as noise added, so that there is a chance that any given individual’s response may not have been recorded [24]. This technique is called two-sided perturbation, while Zeal uses one-sided perturbation (e.g. dummies are added, but no queries are deleted.) Two-sided perturbation may be an effective technique to explore in future PIR work.

Finally, in the shuffle model of DP, LDP is combined with an anonymous communication channel that mixes the data of all users [12]. Zeal implements such a channel to mix the differentially private noise queries with real ones, so Zeal is related to a recent PIR work that uses the shuffle model called ShufflePIR [18] and also to previous work on the "split-and-mix" construction [25]. However, like most PIR schemes, ShufflePIR requires the cooperation of the database, and the scheme in [25] requires both cooperation and database replication. In sum, Zeal inherits the use of a mixnet to amortize costs among users from previous work, and such work is not exclusive to PIR applications, but Zeal’s main extension to these ideas lies in its non-cooperation assumption and the way it leverages untrusted infrastructure.

We summarize the features of existing work in Figure 1, noting how they fail to meet Zeal’s critical requirements.

4 Non-Solutions

There are some alternative approaches to Zeal that may seem intuitive, but we decide against them due either to security issues or due to the non-cooperation constraint.

A naive download split across users. Any uncooperative PIR scheme must eventually hide the intent of users by querying for database records that are unrelated to a user’s intent, as a simple query is the only API available. Based on this, a natural first idea is to simply have a third-party server download the entire database once and use this result to serve any number of user queries. Unfortunately, this only works with a trusted proxy server, which may not be a realistic expectation.

Deterministic dummy generation. In an attempt to reduce the number of superfluous requests and generalize the above non-solution to untrusted infrastructure, one may ask: Could the proxy server generate one dummy per database record and encrypt these indices, ensuring that all database records are queried, while also serving real requests? Unfortunately, the last proxy observes the content of all queries, so if it knows

that one dummy has been created per record, then any indices that appear in more than one query must belong to real clients. This trivially breaks the scheme.

Using the proxy as a cooperative database server. Since the Zeal proxies offer additional functionality beyond simple queries, it is true that we could have them perform existing PIR techniques. However, against a strong adversary with side information, the lower-bound $O(N)$ database download cannot be avoided whatsoever. Due to this restriction, Zeal does not reduce the communication cost of the proxies below $O(N)$. Instead, we accept that proxies have to incur this communication cost due to the dummies, and instead optimize the scheme for clients - a user's communication and computation costs are $O(1)$ in Zeal. Efficient existing PIR schemes for cooperative servers like SimplePIR [23] and PIANO [44] require each user to download large preprocessing hints and these hints must be updated as the database changes. Thus, clients must incur more costs, and asymptotic communication bounds are not strictly better in these schemes due to the bottleneck of the unavoidable database download. Still, a future extension to Zeal may involve incorporating PIR protocols as subroutines to more efficiently transfer dummy and real request data between proxies.

5 The Zeal Protocol

5.1 System Model

Figure 2 shows an overview of Zeal, its architecture, and how the different parties involved in Zeal function at a high level. In Zeal, users do not directly query the *DB*. Rather, they interact with proxy servers. A user request, which is essentially an encrypted integer, is passed forward through an ordered chain of proxy servers toward the *DB*. The proxy servers not only pass encrypted numbers forward; they inject random dummy requests (also called 'differential noise' or 'cover traffic') and mix dummies with real user requests to conceal the intent of users.

Zeal requires one or more proxies and the proxy count does not impact Zeal's security parameters (§6). However, since we assume the presence of one honest proxy at an unknown place in the chain, using more proxies will make this assumption more realistic. (If there is only one proxy, it must be trusted, which largely reduces the novelty of Zeal as a PIR scheme.) The final proxy queries *DB* and all proxies route information back to the users. The database remains unmodified and answers queries from the final proxy identically to how it would in a non-private system.

5.2 Methodology

Zeal's protocol focuses on achieving the following goals, abbreviated as G1 and G2:

- G1. Generate and send fake 'dummy' requests to *DB* that confuse the adversary by obscuring the target user's desired record. However, we should not burden users with the task of generating fake requests so that their communication costs are as low as possible. We instead rely on the increased computational power and bandwidth of proxy servers to generate and send dummy requests to *DB*. This will also have a positive side effect: Dummies will protect all users at once.
- G2. Mix all requests to hide their associated identities - requests from Bob, Alice, and dummy requests should all be indistinguishable. If *DB* knows that dummy requests are dummies, they will ignore them.

Accomplishing these goals while protecting sensitive information from dishonest users, dishonest proxies, and a dishonest database is a major challenge. However, Zeal's construction is surprisingly simple, combining a mixnet with oblivious dummy generation to hide the intent of each individual user among a sea of possibly fake requests.

We must keep in mind the following assumptions, which cannot be avoided for the sake of correctness:

- The first proxy (P_1), responsible for sending query answers directly to users, knows the identity of each user. Thus, it should not know the plaintext indices of any queries, but it may still compute on ciphertexts.
- The last proxy (P_{final}), responsible for routing queries to *DB*, knows the exact index of such queries. Thus, it should not know the associated identities of its queries.

We will leverage the existence of the honest proxy P_h at an unknown spot in the chain. Adversarial parties before P_h , including users and proxies, will only know the identity associated with each request. Adversarial parties after P_h , including proxies and the database, will only know the content of each request. These two groups will not be able to cross-reference information to recover the target user's intent. First, we explain how requests are encrypted to facilitate this process, and then we will dive into the job of the proxies.

5.3 Phase 1: Request Generation

Again, the target user Bob starts with a raw request containing an index $i \in [0, N)$ of the database array. The request cannot simply be encrypted using one public key anymore - we do not know which proxy can be trusted with this keypair. Naturally, Bob must encrypt his index once per proxy, ensuring that the plaintext index is only revealed once P layers of encryptions are removed. For this task, we use onion routing [35].

Each proxy P_x where $x \in [1, P]$ announces a unique public key, $PPub_x$, and keeps a corresponding private key $PPriv_x$. For each of these, Bob generates a keypair: $UPub_x, UPriv_x$. Together, using Diffie-Hellman, this enables Bob to generate

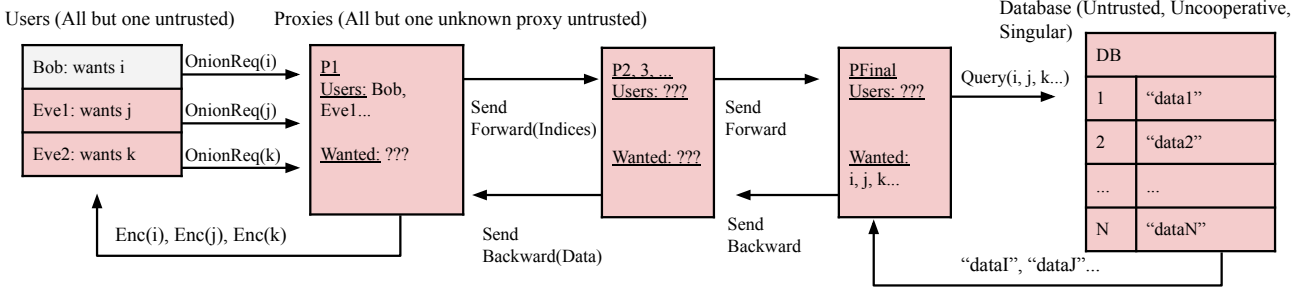


Figure 2: An overview of the Zeal system. Untrusted proxies forward and shuffle both real and dummy onion-encrypted requests to the untrusted database and then pass back encrypted data records.

Message Field	User's Public Key 1	User's Public Key 2	User's Public Key 3	Index
AES Encryptions (applied top down, decrypted bottom up)	None	k_1	k_2 k_1	k_3 k_2 k_1

Figure 3: The Zeal structure of one Zeal request when $P = 3$

P symmetric AES keys, which we refer to as k_1, k_2 , etc. For all symmetric keys k_x :

$$k_x = \text{Hash}(\text{DiffieHellman}(PPub_x, UPriv_x)) \quad (1)$$

Imagine a simple naive routing algorithm Bob could use with these keys: He could simply encrypt his index i with each symmetric key, announce each public key to its corresponding proxy, and then send the index to P_1 . Each proxy could compute their corresponding k_x , decrypt the index once, and send it onward. This would certainly hide the index until P_{final} decrypts the final layer, but this strategy has a problem: Bob's request cannot ever be anonymized. In order to decrypt his index field, each proxy has to compute Bob's unique symmetric key k_x and use it on Bob's unique message only. This would require the proxies knowing the identity of Bob's request, as his public keys are separate from the message (which is just an index.)

In reality, user requests are formulated according to Figure 3, which shows the message structure for 3 proxies. All public keys and the request index are concatenated:

$$\text{OnionReq}(i) = (UPubCtxt_1 \dots UPubCtxt_P, \text{EncIndex}) \quad (2)$$

The first public key need not be protected, since P_1 knows Bob's identity anyway, but all further public keys are themselves encrypted by the symmetric keys of all previous proxies. For any proxy number $x \in [1, P]$:

$$UPubCtxt_x = \text{Enc}(k_1, \text{Enc}(k_2 \dots \text{Enc}(k_{x-1}(UPub_x)))) \quad (3)$$

For example, if $x = 3$, the public key $UPub_3$ will be encrypted by k_1 and k_2 . Thus, P_3 will only be able to see it once P_1 and P_2 have removed these encryption layers.

As for the index, it will remain encrypted until all proxies have decrypted their layers:

$$\text{EncIndex}(i) = \text{Enc}(k_1, \text{Enc}(k_2 \dots \text{Enc}(k_P(i)))) \quad (4)$$

Any proxy P_x will see $UPub_x$ as a plaintext, compute k_x , and decrypt all other fields in the message with k_x , including the index. When P_x sends the message to the next proxy, it discards $UPub_x$ from the message entirely. Thus, the relevant content of the message (the index) is hidden until it reaches the final proxy, and Bob does not have to explicitly announce his public keys to any proxy, so his identity may still be hidden.

All but one proxy at an unknown place in the chain will reveal their secrets, including k_x , to the adversary, but this is not an issue. Bob's index cannot be immediately decrypted unless the adversary has *all* P keys. The symmetric key corresponding to the honest proxy P_h will only be computed by P_h and will be removed from future message fields, so the honest proxy protects this value and by extension Bob's request.

Algorithm 1 User: Send(i)

```

OnionRequest = [i], SymmKeys = []
for (index i, Proxy P) in Chain do
    (PublicKey, PrivateKey) = Keypair()
    Compute and store symmetric key in SymmKeys[i]
    Append symmetric key to OnionRequest
end for
for index i in Chain, reverse order do
    for index  $j \leftarrow i$ ,  $j < \text{len}(\text{Chain})$ ,  $j++$  do
        Encrypt index field and jth key with SymmKeys[i]
    end for
end for
SendToP1(OnionRequest)

```

5.4 Phase 2: Forward Pass

The first proxy, P_1 , immediately knows $UPub_1$, which is not encrypted. P_1 uses $UPub_1$ to generate k_1 :

$$k_1 = \text{Hash}(\text{DiffieHellman}(UPub_1, PPrivate_1)) \quad (5)$$

P_1 then saves k_1 and discards the message field corresponding to $UPub_1$ (this field is not relevant to other proxies.) P_1 uses k_1 to decrypt all remaining fields within the message. Note that, at this point, $UPub_2$ is no longer encrypted at all. When the message is sent to P_2 , the same process can be repeated. This continues until the index itself is decrypted by P_{final} .

Next is the issue of anonymity. We need to mix the requests from all users to anonymize them. So far we have been discussing one request only, but this process can occur in parallel with many requests at once. Each request has its own associated identity, index, and keys.

Again, P_1 shuffles the relative order of the packet with a random permutation π_1 and stores the permutation used as an integer array. Since the content of the requests appears random and their order is random, P_1 's output appears completely unrelated to its input unless we know its private variables (the recovered symmetric keys and its permutation array). This means that we achieve G2 after the honest proxy sends forward requests, which will always happen before they have been completely decrypted.

Algorithm 2 Proxy: SendForward(Packet)

```

NewPacket = [], SymmKeys = []
 $\pi$  = RandomPermutationArray()
for once per dummy needed do
    Packet.append([key1 = randomebits() ... encIndex = randomebits()])
end for
for req in Packet do
    Compute symmetric key and store it in SymmKeys
    Remove key field used
    Decrypt all fields with this symmetric key
    Append result to NewPacket
end for
NewPacket.Shuffle( $\pi$ )
SendToNextProxy(NewPacket)

```

Once the requests are both decrypted and shuffled they are sent to the next proxy and Algorithm 1 repeats. By the time the final proxy, here P_3 , receives the ciphertexts, it recovers k_3 and uncovers the raw indices of the requests. At this point, P_3 alone queries DB , retrieving all needed data. (The last line of pseudocode in Algorithm 2 would instead be queries to DB .) This strategy means that the proxies must maintain state while executing the Zeal protocol.

5.5 Phase 3: Backward Pass

At this point all queries have been fulfilled anonymously. Data must be sent backwards to the users. Again, the retrieved data must be obscured on the return trip: If P_1 is an adversary and sends raw query answers to users, the scheme will be broken, since P_1 would know which record each user wants.

In the forward pass, indices were encrypted with one corresponding key per proxy. Naturally, the database records can be encrypted with the same aforementioned AES keys k_x that have already been recovered. This means that the retrieved data gains layers of encryption as it travels back to the user, eventually being encrypted by the same keys in the same order as the original Index field in Figure 3. When a user receives a ciphertext data point, they decrypt all layers with their personal AES keys. Note that the return trip requires no Diffie-Hellman operations, so its computational cost is lower than the forward pass, but the messages must contain entire records rather than integers, so its communication cost is greater.

We also need to gradually unveil the intended recipient of each data point. Similarly to how the backwards pass reverses the encryption process, it also reverses the shuffling process. The packet of records is unshuffled according to π to assure that it will be unshuffled into its original order upon reaching P_1 . This ensures that the correct record is routed to each user.

Algorithm 3 Proxy: SendBackward(Packet)

```

Packet.Unshuffle( $\pi$ )
for (data, symmKey) in (Packet, SymmKeys) do
    Encrypt singular data field with symmKey
end for
if First Proxy then
    Route results to users according to original order
else
    SendToPreviousProxy(Packet)
end if

```

Since an honest proxy at some point in the chain hides its shuffling permutation (e.g. conceals identity per request) and set of keys (e.g. conceals the contents of requests) and does not send them to any other party, the chain of proxies emulates a single trusted proxy that reveals neither identity nor intent.

We have accomplished G2. To accomplish G1, we can in fact reuse everything we just came up with to hide partial information from proxy servers. We can in fact generate dummy requests at the proxy level, rather than at the user level, so that users will not have to download extra content from DB .

5.6 Dummy Requests

P_1 must generate many dummy requests that refer to uniformly random indices in the database array. If the proxy was

trusted, then we would not need any special process to generate them. However, since P_1 is untrusted, it cannot generate plaintext indices. P_1 must generate dummy requests in such a way that it does not directly know their index contents.

This is accomplished with a subtle difference between real and dummy requests: While Bob started with a plaintext index and generated a ciphertext using public keys, dummy requests are generated *obliviously* by P_1 . A dummy has completely random bits for its index field as well as its public key fields.

Thus, P_1 generates dummy requests and knows they are fake, but an adversary monitoring P_1 has no idea what actual indices they refer to, since the random bits form a ciphertext for which the keys are not all known. This idea is unintuitive, but it has the same justification as the assertion that the adversary cannot immediately decrypt Bob’s request when P_1 receives it. A ciphertext appears random to those who do not have the keys to decrypt it. Thus, distinguishing between truly random bits and a genuinely encrypted ciphertext is just as hard as decrypting such ciphertext.

By the time dummy requests are received by the final proxy and their true index is revealed, the adversary no longer knows what identity they are attached to, and cannot tell that they are fake. Once the dummy requests are fulfilled and their data is returned to P_1 , there will be no way to decrypt the data, since one of the necessary AES keys will be unknown.

Thus, if we generate a dummy request whose real index is the possible index of (i, j) that Bob did not choose, the adversary will have no idea which came from Bob and which came from the dummy. Since all requests are relatively anonymous, the only situation in which the target user’s intent is revealed is when the unwanted index (i, j) is never generated. We can make the chance of this occurring very low by increasing the number of fake requests sent. We quantify the amount needed in a later section (§6.3).

This technique provides two novel contributions: First, it allows the threat model to allow for all but one user to be untrusted without incurring any additional costs, as all dummy generation is offloaded to untrusted infrastructure that is independent of the users. This strengthens the threat model when compared with Toledo’s schemes [37]. Second, it means users do not have to download or create dummies.

In addition to designing Zeal’s protocol, we prove rigorously that information revealed before and after P_h cannot be combined to compromise the target user. In fact, adversaries cannot even tell the difference between a world where the honest proxy follows protocols correctly and a world where the honest proxy creates an entirely new and random request packet. The full proof is included as a supplementary document. We reason that before the honest proxy, when content is obscured, the adversary sees only random bits. After the honest proxy, an unknown shuffling permutation has been used, so the adversary sees anonymized data. The pseudorandomness of decryption and encryption ensures that these two periods cannot be related. We strip down the Zeal setup to a

database of only two indices and one user to make this process clearer, eliminating variables that the adversary already ignores (i.e. non-relevant indices and the actions of dishonest users.)

There is one significant but simple implementation-level optimization we can make to Zeal that reduces the overhead introduced by Diffie-Hellman operations. This optimization is called batching, and essentially involves grouping multiple requests under the same public-private keypairs, so that each Diffie-Hellman operation can encrypt and decrypt multiple records. This optimization is described in detail in an appendix (§Appendix B).

6 Privacy Guarantee

6.1 Definitions

Zeal uses **differential privacy** [4, 37] to specify the level of security offered to users. The adversary learns some information about the target user’s intent when observing Zeal, but we can precisely bound the leakage.

Specifically, a PIR scheme is defined as (ϵ, δ) private if the following probabilistic equation always holds:

$$P[O|Q_i] \leq e^\epsilon * P[O|Q_j] + \delta. \quad (6)$$

Here, $P \in [0, 1]$ refers to the probability of an event. O is the sequence of all observations made by all adversarial parties during a single end-to-end execution of the PIR protocol. We model O as the output of the PIR scheme to adversaries: A combination of the final sequence of indices observed by the final proxy and the real content of all queries made by compromised users; the adversary observes an ordered list of integers in the range $[0, N)$ that represent both real and dummy queries. Since all non-targeted users are dishonest and the adversary may control them, the security definition is equivalent whether or not we consider the requests of these users. Thus, we assume that all integers in O other than the target user’s real query are randomly generated dummy requests.

Semi-honest proxies certainly reveal other information to the adversary, including their private keys, shuffling permutations, and symmetric keys generated by Diffie-Hellman. However, in Zeal’s security proof, we show that none of this information is useful and that all information learned by the adversary can be captured with the sequence of indices generated by the final proxy. In the proof, we reason that all variables generated by the Zeal proxies and users are independent of query indices except for the query indices themselves. We use this fact, along with the addition of dummies, to show the indistinguishability of two worlds in which the user chooses two different indices.

Q_i or Q_j are two substituted events where the target user picks index i or j respectively [38]. We focus on two indices only because, as mentioned in the threat model (§2.1), we

assume the adversary has the maximal amount of side information about the user and has narrowed down their choice to two possibilities.

Conceptually, ϵ is the factor that determines how much confidence an adversary might gain that the user did pick one index over another. For example, if $\epsilon = \ln 2$, the adversary will at worst become twice as confident that the user chose one index over the other, relative to before the PIR scheme initiated. This allows us to capture security regardless of the adversary's prior confidence about the user's actions - the definition is the same whether the adversary has no idea whether the user chose i or j or whether they are already 99% confident the user chose i over j .

δ is the probability of a leakage event in which ϵ -privacy does not hold. These events may represent the adversary discovering the user's intent with certainty or the adversary simply gaining more confidence than the value of ϵ would suggest. In the case that $\delta = 0$, the leakage of the scheme is precisely bounded because the value of ϵ always holds. In Zeal, $\delta > 0$, so during some rare events, we cannot bound the information leakage of the scheme, but by adjusting Zeal's parameters, we can set δ to be arbitrarily low.

6.2 Parameter Calculation

ϵ and δ are expressed as a function of the following variables:

- Database of size N .
- w dummies are generated.
- The target user is known to have chosen one of two known indices: i or j . For simplicity, we will assume throughout this section that the user has chosen i , but this choice is arbitrary.
- Zeal generates dummies at random. Let X_i be the random variable representing the number of dummies that have been generated for index i , and X_j for index j . Dummies may also be generated for other indices, but these are not relevant in the calculations.

First, we focus on the calculation of δ .

If the target user chooses i , and no dummies are generated for j e.g. $X_j = 0$, then the adversary trivially discovers the target user's intent because O will not contain any references to j . In this situation, no finite ϵ satisfies the equation because $P[O|Q_j] = 0$. Thus, $\delta \geq P[X_j = 0]$.

For each dummy, the probability that the dummy will not choose j is $(1 - 1/N)$. All w dummies are independent, so the probability that none of them choose j is $(1 - 1/N)^w$. Therefore,

$$\delta \geq (1 - 1/N)^w \quad (7)$$

For a given database of size N , we can achieve any desired value of δ by adjusting w . Solving the above equation for w

yields

$$w \geq \frac{\ln(\delta)}{\ln(1 - 1/N)} \quad (8)$$

For any given δ , $w \in O(N)$. Note that the w dummies protect all users at once. We do not have to generate w dummies per user.

If $\delta > P[X_j = 0]$, then this suggests some other event has been included in the δ probability mass, and we discuss this shortly. First, we can calculate the value of ϵ assuming we set δ to the minimum value of $(1 - 1/N)^w$. ϵ must upper-bound the ratio $\frac{P[O|Q_i]}{P[O|Q_j]}$ and vice versa for all possible values of O other than those included by the δ probability. The highest ratio, e.g. the worst case where $P[O|Q_i]$ is much higher than $P[O|Q_j]$, is the case where $X_j = 1$ and $X_i = w - 1$. Comparing the probabilities:

$$P[O|Q_j] = P[X_i = w] = \frac{1}{N^w} \quad (9)$$

Whereas

$$P[O|Q_i] = P[X_i = w - 1 \cap X_j = 1] \quad (10)$$

$$= P[X_i = w - 1] * P[X_j = 1 | X_i = w - 1] \quad (11)$$

$$= (w * \frac{1}{N^{w-1}}) * (\frac{1}{N}) = \frac{w}{N^w} \quad (12)$$

This means that the ratio between the two probabilities is w , so $\epsilon = \ln(w)$. In this (worst) case, $X_i = w - 1$ and $X_j = 1$, but a similar calculation could be done for any X_i and X_j , and generally:

$$\frac{P[O|Q_i]}{P[O|Q_j]} = \frac{X_i + 1}{X_j} \quad (13)$$

If we only care about minimizing δ , then $\delta = (1 - 1/N)^w$ and $\epsilon = \ln(w)$. However, this does not result in a good security guarantee: For example, if we require that $\delta = 10^{-4}$, then $w = \frac{\ln(10^{-4})}{\ln(1 - 1/N)} \approx 9.2N$, so $\epsilon \approx 9.2 * \ln(N)$. This results in very high values of ϵ : For even a small database with $N = 10^4$, $\epsilon \approx 11.43$, making for a very weak security guarantee. This is especially inefficient considering that the worst-case scenario that produces such a high ratio, e.g. $X_i = w - 1$, occurs only with a negligible probability.

To achieve a better tradeoff, we can include low-probability events that result in high (but not total) leakage in the value of δ . δ always includes the event where $X_j = 0$, but we can set ϵ to any value as long as we add the probability of events where $\frac{X_i + 1}{X_j} \geq e^\epsilon$ to δ . Specifically, given the values of w and N ,

$$\delta(\epsilon) = P[X_j = 0] + \sum_{k=1}^w (P[X_j = k] * P[\frac{X_i + 1}{k} > e^\epsilon | X_j = k]) \quad (14)$$

The first term is the lower bound for δ . The summation term is the sum of the probabilities of all events where the ratio between i and j dummies exceeds the ratio allowed by ϵ . First, we expand the first term in the summation. There are $\binom{w}{k}$ ways to make $X_j = k$, and to make X_j exactly k , w random uniform draws from $[0, N)$ must result in j and $w - k$ draws must result in some other index. Therefore,

$$P[X_j = k] = \binom{w}{k} * \left(\frac{1}{N}\right)^k * \left(1 - \frac{1}{N}\right)^{w-k} \quad (15)$$

We can use this same result to expand the second term in the summation, e.g. to expand $P[\frac{X_{i+1}}{k} > e^\epsilon | X_j = k]$. The expression $\frac{X_{i+1}}{k} > e^\epsilon$ is equivalent to $X_i > e^\epsilon * k - 1$. Since the condition $X_j = k$ guarantees that $X_i \leq w - k$, the probability of this event is

$$P[X_i > e^\epsilon * k - 1 | X_j = k] = \sum_{t=\lceil e^\epsilon * k \rceil}^{w-k} P[X_i = t] \quad (16)$$

This further expands to

$$\sum_{t=\lceil e^\epsilon * k \rceil}^{w-k} \binom{w-k}{t} \left(\frac{1}{N-1}\right)^t \left(\frac{N-2}{N-1}\right)^{w-k-t} \quad (17)$$

In sum, we can choose any required ϵ , and then scale the number of dummies, w , until the required value of δ is reached. δ has a concrete lower bound as a function of N and w , while ϵ does not. The more dummies generated, the lower we can make both parameters.

6.3 Choosing ϵ , δ , and w in Practice

Zeal uses equation 14 to decide on a value of w and thus determine how many dummies it will add to each packet of user requests at runtime. To do so, an administrator provides desired values of ϵ and δ to Zeal, and based on a known value of N , we iteratively try different w values to achieve the desired δ and ϵ using a simple yet accurate estimation algorithm described in an appendix (§Appendix A).

Figure 4 shows different values of ϵ and δ and the dummy requests required as a factor of N . The green cells represent security parameters and dummy counts that are feasible for performance. For example, for $(\epsilon, \delta) = (2, 10^{-3})$, the value of 9.53 means that, to achieve those parameters, we must generate 9.53 dummies per database record.

$\delta \setminus \epsilon$	0.1	1	2	5	10
10^{-2}	45.26	14.39	5.71	4.61	4.61
10^{-3}	45.26	24.41	9.53	6.91	6.91
10^{-4}	45.26	34.62	13.36	9.21	9.21
10^{-5}	45.26	44.99	17.15	11.51	11.51

Figure 4: Dummies needed, as a factor of the database size N , for required δ and ϵ values.

Note that, for very small values of δ and very large values of ϵ , the other parameter stops being relevant. This is because the lower bound on δ either dominates the equation completely or shrinks to being negligible.

6.4 Information Leaked in Practice

The DP security definition uses a worst-case analysis. For example, for a value of $\epsilon = 2$, this indicates the adversary may become $e^2 \approx 7.38$ times more confident that the user chose one index over another. This seems quite high, but in practice, the average amount of information leakage is incredibly low. In figure 5, we consider one user who uses Zeal and repeatedly queries for the same index, $i = 0$, out of a database with 10^6 items. We arbitrarily set $j = 1$. We monitor Zeal's access pattern with $w = 13.36 * N$, so $\epsilon = 2$ and $\delta = 10^{-4}$, recording the ratio between the occurrences of i and j in the observation sequence O . We repeated this trial 1000 times. Indeed, in the worst case, we observed a ratio of 12 (i.e. 12 accesses to record i and only one access to record j), which is classified as a δ event, since $12 > e^\epsilon$. However, the average ratio is only 1.26, the median is 1.25, and the worst case is a single outlier trial, meaning that Zeal leaks very little information in the vast majority of cases.

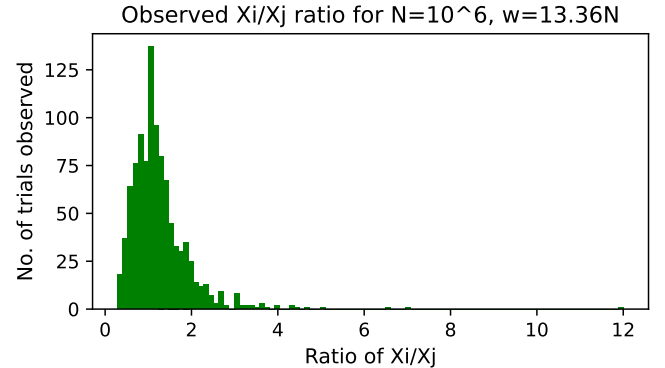


Figure 5: The ratio of accesses between arbitrarily chosen records i versus j across 1000 trials of Zeal.

6.5 Repeated Requests

The value of ϵ implies what level of security is afforded to a user who makes multiple requests over time: Suppose that, in a sequence of multiple Zeal rounds where all parameters remain the same, the user chooses index i every time. The adversary will become at worst e^ϵ times more confident about the user's choice each time. Thus, after three rounds, the adversary will become at worst $e^{3\epsilon}$ times more confident, as all Zeal rounds are independent of one another. In general, this means that differential privacy is less attractive for users

who make repeated queries to a record or group of small records over time, but even in these scenarios and with a strong adversary, users have precise guarantees on how much information is leaked.

6.6 Improvement Over the Naive Solution

A key challenge of Zeal is that the number of dummies needed to reliably obscure the intent of users plus padding indices, w , exceeds the size of DB . This is understandable considering that they are generated randomly and that we are hoping to cover every index in the database. On the surface level, this may seem to indicate that Zeal is worse than the naive solution of downloading the entire database. However, the two large advantages of Zeal are that the user itself does not have to download any extra data themselves and that any number of users benefit from the same w dummies at once.

From the user’s standpoint, this certainly reduces overhead below that of the naive solution, but we can also intuitively see that the database would also see improvement. This is because, for the naive solution, each user downloads the database, so DB must retrieve and send $U * N$ records. Even though $w > N$, for realistic scenarios, $w < U * N$ by a wide margin.

For example, for $(\epsilon, \delta) = (2, 10^{-3})$, the number of dummies needed is around 10 times the size of DB (§6.3). Therefore, only about 10 users must be querying DB (and/or waiting for their results) simultaneously for Zeal to lower DB ’s communication and computation overhead below that of the naive solution. By extension, the users will receive their results much faster, as the database’s resources will not be nearly as strained. We confirm this intuition in our evaluation (§8.4), but first, we focus on Zeal’s implementation.

7 Implementation Details

We implement Zeal in Go with about 2700 lines of code. Zeal is built to run on a chain of AWS EC2 servers: One for the database, one per each proxy, and one load generator that simulates the presence of all clients (though it also supports multiple users operating their own devices.) The Zeal implementation separately implements code for all three parties, but the database is deliberately simple, structured as a non-encrypted integer-indexed array that can be queried freely. We use cryptographic primitives from Go’s crypto library. Specifically, we use the Curve25519 package for fast Diffie-Hellman operations [8]. We implement simple database calls and communications between users and proxies with Go’s net/rpc library.

7.1 Proxies

Parameters. All proxies share the same highly configurable code. To start a Zeal proxy and make the service available to clients, we provide 5 parameters: The role of the proxy (first,

last, or somewhere in the middle (§5)), the size of the database N , the batch size B discussed in the appendix (§Appendix B), the total number of proxies P , and the IP address of the next proxy in the chain. The proxies then become persistently available, waiting until users have submitted requests.

Rounds. In the real world, users do not all simultaneously submit queries to a public database, but Zeal still needs to amortize its costs over as many users as possible. Zeal therefore runs in fixed-length ‘Rounds’, where proxies wait briefly while users queue requests. The first proxy, P_1 , coordinates these rounds and triggers the proxy chain to begin routing requests to the database. This mechanism is entirely internal to the proxies: users simply submit requests and later receive a response without knowledge of the round system. For efficiency, Zeal pipelines rounds, see Appendix C.

Query Deduplication. P_{final} always observes multiple requests per round, real and/or dummy, that correspond to the same database record. When P_{final} queries the database, it only ever downloads each record once, so at most, it downloads the entire database once, copying records to fulfill all requests. This improves efficiency, but it does not improve Zeal’s security guarantee, as the number of requests associated with each record is still visible to the final proxy, which may be untrusted.

7.2 The Database

Records. Our simple database is locally stored as a lengthy text file containing strings. The strings are indexed by their integer line in the file. For our experiments, we set the length of these strings as 1000 characters, meaning the size of each record is 1 Kilobyte. This text file is the only parameter needed by the database.

Supported Operations. The only operations supported by the database are a single-record query and a multi-record query. It performs no cryptographic operations and does not inherently support any PIR schemes.

7.3 The Users

Client interface. In order to use Zeal, a client must receive public keys from all proxies and use them to onion-wrap their chosen index. They then send this ciphertext to the first proxy and open a listener channel to receive the result from P_1 . It is not the user’s job to generate dummies or contact the database directly.

Code is available for an individual client, but our experiments simulate many clients at once.

Simulating many users. To perform experiments, we implement a simple yet customizable load generator for Zeal. It automatically generates requests, checks results for correctness, and aggregates latency and network bandwidth data. The load generator uses a Poisson distribution to randomly distribute requests with a consistent average total.

Ensuring chosen indices are valid. Due to oblivious dummy generation, we must ensure that any unsigned integer is a valid index for any database, as we cannot predict the plaintext contents of dummies when they are created. However, real databases have a limited number of records. To fix the issue, any integer k is converted into a valid index from $[0, N)$ with the modulo operation $k \leftarrow k \% N$. Request indices are represented as 64 bit unsigned integers in our implementation, so any combination of bits that has been obviously generated can be reduced in this way without error.

To ensure dummies and real requests are indistinguishable, the user randomly increases their chosen index i to possibly be larger than the database size. If r is a random integer, then $i \leftarrow i + r * N$. This means that the client must know N , but N is already publicly available.

If N changes during runtime, the proxy will use the old value of N to divide the request index, ensuring each query’s original intent is preserved. If N has increased since query submission, no dummies need to be generated for the new records, as the adversary already knows that no real users were interested in these records - they did not exist at the time of query submission. If N has decreased, some requested records may be unavailable, but this has nothing to do with the intent of any user.

8 Evaluation

In this section, we will evaluate Zeal’s performance in terms of the users’ end-to-end latency and compare this data with the naive solution of each user downloading the entire database. We also monitor the network bandwidth usage of each user in each experiment. A summary of our results is as follows:

- For a database with 1 million records of 1 Kilobyte each and 10^4 active users, Zeal’s latency is about 3 minutes. This improves upon the naive solution’s latency by a factor of 50. When this same experiment is run with 10^5 users, Zeal’s latency is still under 4 minutes, while the naive solution crashes due to resource limits.
- For all databases, the extra network overhead for Zeal users (i.e. bits of data downloaded beyond that of the records themselves) is only 700 bits. This improves upon user network bandwidth usage for the naive solution by a factor of about N - for 1 million records, the improvement factor is about 1 million.

8.1 The Baseline

No existing research presents a nontrivial PIR scheme satisfying our assumptions and threat model, so the only useful baseline to compare against Zeal is the naive solution where each user individually downloads the entire database. We implement a load generator with which we test the latency

of this solution. This load generator ensures correctness and aggregates latency and bandwidth data identically to the Zeal load generator (§7.3).

8.2 Experiments Run

We run three main experiments, focusing on the size of the database N , the user count U , and the proxy count P :

- E1. We change the number of records in the database, N , to see its effect on Zeal’s end-to-end latency and that of the baseline. In this experiment, $U = 10000$ and $P = 3$.
- E2. We change the number of users actively using Zeal or the baseline, U , to measure the same statistics. In this experiment, $N = 10^6$ and $P = 3$.
- E3. We vary the number of proxies used in Zeal’s chain, P , to see its effect on latency. For this experiment, the baseline is not tested, as it does not use any proxies. In this experiment, $U = 10000$ and $N = 10^6$.

Some Zeal parameters are kept constant throughout all experiments. The waiting period between Zeal rounds is set at 5 seconds. The database, the load generator, and all the proxies have 25Gbps bandwidth. The inter-request period of each user is, on average, 30 seconds, simulating a situation where users view some web resource and wish to retrieve a new one after around 30 seconds of browsing. Users submit requests for 60 seconds, but data is still collected afterwards. These parameters are all the same for the baseline, except that the baseline does not use synchronous rounds. Lastly, the δ and the ϵ are also kept constant at $\delta = 10^{-4}$ and $\epsilon = 2$. We keep (ϵ, δ) fixed across experiments because they define Zeal’s privacy guarantee; varying N, U , or P should not change the privacy budget, so we instead scale $w(N, \epsilon, \delta)$ to preserve the same guarantee and measure performance under that constant security target.

8.3 Testbed

We represent both proxies and the database server with AWS. The machine type is r5.metal. These machines use 96 vCPU cores, x86_64 architecture, 256GiB of RAM, and up to 25Gbps network bandwidth. All instances are located in the US West (N.California) region on AWS.

8.4 Results

In Figure 6, we depict the results of E1, where we increase the size of the database from 10,000 records to 5 million records. The naive solution crashed for the final test due to resource limitations, and for $N = 10^6$, the test could only run for 15 seconds without crashing. As expected, increasing the size of the database significantly increases Zeal’s latency because the

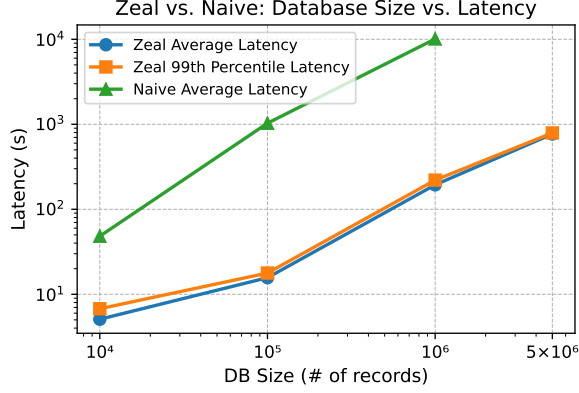


Figure 6: Database size N versus latency

number of dummies generated, processed, and sent, y , scales roughly linearly with the size of the database.

Zeal's latency is better than that of the baseline for all database sizes, but the difference grows much larger as we scale up the size of the database. For $N = 1$ million records, Zeal's average latency is a little over 3 minutes, but the baseline's latency is almost 3 hours. This 50x speedup is expected considering that, while Zeal's total amount of queried records is about $13.36 * N$ (§6.1) when all users query for one record, the baseline makes a total of $10000 * N$ queries in the same situation, as all 10000 users request all N records each. The reason that the speedup is not close to $10000/14 \approx 729$ is that the proxies introduce additional communication overheads (all data from the database must be sent to each link in the chain) and CPU overheads (encryption, decryption, and shuffling.)

Network communication between proxies is the primary latency bottleneck as most computational steps can be parallelized with goroutines. In both Zeal and the baseline, the users' bandwidth is not a significant factor, as both solutions strain the database's bandwidth until the effective bandwidth it has available per query is quite low.

In Figure 7, we report the results of E2, where we vary the number of users from one user to 100,000 users. Again, the naive solution could only be run for 15 seconds for the second-to-largest trial and crashed for the final one. For $U = 1$ and 10, Zeal is indeed slower than the naive solution. This is expected - Zeal's design revolves around amortizing its costs among all users simultaneously, and adding more users does not affect the number of dummies Zeal must create. However, as U increases, the naive solution does not scale, for the same reason that it did not scale in E1 - The database must service $N * U$ queries. For $N = 10^3$, Zeal is 7x faster than the naive solution, and the naive solution cannot handle greater user counts without the experiment being shortened.

Finally, we report the results of E3 in Figure 8, where we change the number of proxies in the Zeal chain from 3 to

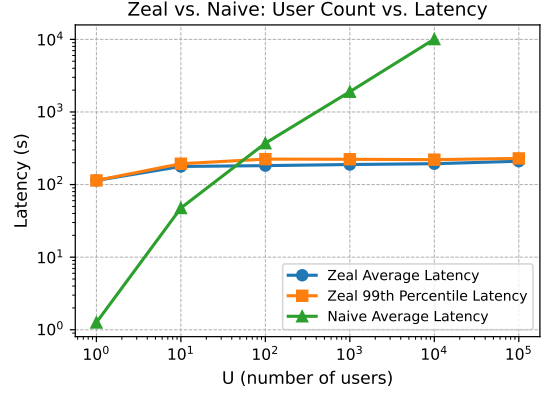


Figure 7: User count U versus latency

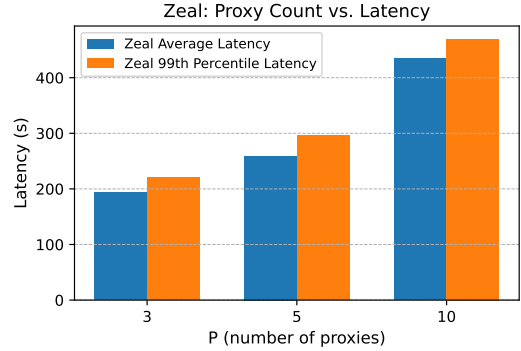


Figure 8: Proxy count P versus latency

5 and finally to 10. Latency scales linearly with P . This is expected: Each proxy does roughly equal work, as the number of requests in the packet that must be shuffled, decrypted, sent, and routed back to users is the same at all points in the chain.

Increasing the number of proxies changes the number of onion layers each request is encrypted with, but Zeal's security parameters (δ , ϵ , and the number of dummies generated) are unchanged. Thus, increasing the number of proxies is only beneficial because it makes the assumption that one proxy can be trusted more realistic.

The network bandwidth used by individual clients is universally better for Zeal than for the naive solution in all experiments and trials. For $P = 3$, Zeal's cryptographic overheads force users to download 700 bits of extra data. The records themselves are 8Kb (e.g. 8 kilobits or 1 kilobyte), but users download 8.7 Kb. However, for the naive solution, users download the entire database. For $N = 10^5$, this is 80 Mb, or for $N = 10^6$, it is 800 Mb. Thus, clients use dramatically less network resources when choosing Zeal.

9 Conclusion

Existing PIR research has not yet reckoned with the existence of databases that do not cooperate with protocols beyond servicing simple queries. Schemes that work without database cooperation are either trivial or rely on other strong assumptions such as the presence of multiple servers, the presence of a trusted proxy or multiple trusted users, or a weak adversary. Zeal is the first nontrivial PIR scheme that functions without these assumptions. It relies on a system of proxy servers that generate dummy requests and shuffle them together with real requests from any number of users to provide relative anonymity. Clients need not download any dummy requests or preprocess data from the database. Only one out of any number of proxies must be trusted to provide security against a semihonest adversary with side information about the client. Zeal already improves upon the naive solution dramatically, but design-level optimizations to Zeal are needed to make it widely applicable to internet browsing.

Acknowledgments

We thank the anonymous reviewers for their detailed feedback. Other acknowledgments have been omitted for anonymity.

Ethical Considerations

All experiments in this paper were done using a simulated database under our control. No external APIs were used, so there were no terms of service to consider. However, when applied to real databases online, it is important to note that, since Zeal sends a huge volume of requests, such requests may need to be batched or spread out to conform to rate limiting and the terms of service for the specific database used.

Open Science

Zeal's proof can be accessed here: https://docs.google.com/document/d/1XffQx9Crgkpad_WcE5Z4lhdRlMcqsa0_P9w15TQSxYw/edit?tab=t.0. Zeal's source code, managed on GitHub, unfortunately cannot be made available at this time because the software is proprietary. We intend to release the source code to the public as soon as we are allowed to do so.

References

- [1] Jayadev Acharya, Keith Bonawitz, Peter Kairouz, Daniel Ramage, and Ziteng Sun. Context-aware local differential privacy, 2020. URL: <https://arxiv.org/abs/1911.00038>, arXiv:1911.00038.
- [2] Carlos Aguilar-Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. XPIR: Private information retrieval for everyone. *PETS*, 2016(2):155–174, 2016.
- [3] Ishtiyaque Ahmad, Divyakant Agrawal, Amr El Abbadi, and Trinabh Gupta. Pantheon: Private retrieval from public key-value store. *Proc. VLDB Endow.*, 16(4):643–656, dec 2022. doi:10.14778/3574245.3574251.
- [4] Kinan Dak Albab, Rawane Issa, Mayank Varia, and Kalman Graffi. Batched differentially private information retrieval. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3327–3344, Boston, MA, August 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/albab>.
- [5] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. Pir with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 962–979, 2018. doi:10.1109/SP.2018.00062.
- [6] Shariq Bashir and Akmal Saeed Khattak. Private web search using proxy-query based query obfuscation scheme. *IEEE Access*, 11:3607–3625, 2023. doi:10.1109/ACCESS.2023.3235000.
- [7] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers computation in private information retrieval: Pir with preprocessing. In Mihir Bellare, editor, *Advances in Cryptology — CRYPTO 2000*, pages 55–73, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
- [8] Daniel J. Bernstein. Curve25519: New diffie-hellman speed records. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *Public Key Cryptography - PKC 2006*, pages 207–228, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [9] Benjamin M. Case, James Honaker, and Mahnush Movahedi. The privacy-preserving padding problem: Non-negative mechanisms for conservative answers with differential privacy, 2021. URL: <https://arxiv.org/abs/2110.08177>, arXiv:2110.08177.
- [10] David L. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Commun. ACM*, 24(2):84–90, Feb 1981. doi:10.1145/358549.358563.
- [11] Zehao Chen, Honghui You, Qian Wei, Hang Lu, Lei Ju, and Zhaoyan Shen. *SmartPIR: A Private Information Retrieval System using Computational Storage Devices*, page 1749–1762. Association for Computing Machinery, New York, NY, USA, 2025. URL: <https://doi.org/10.1145/3725843.3756060>.
- [12] Albert Cheu, Adam Smith, Jonathan Ullman, David Zerber, and Maxim Zhilyaev. Distributed differential privacy via shuffling. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019*, pages 375–403, Cham, 2019. Springer International Publishing.
- [13] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. *Journal of the ACM (JACM)*, 45(6):965–981, 1998.
- [14] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: the second-generation onion router. In *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13, SSYM’04*, page 21, USA, 2004. USENIX Association.
- [15] Josep Domingo-Ferrer, Agusti Solanas, and Jordi Castellà-Roca. h(k)-private information retrieval from privacy-uncooperative queryable databases. *Online Information Review*, 33:720–744, 08 2009. doi:10.1108/14684520910985693.
- [16] Cynthia Dwork. Differential privacy. In Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener, editors, *Automata, Languages and Programming*, pages 1–12, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [17] Arik Friedman and Assaf Schuster. Data mining with differential privacy. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’10*, page 493–502, New York, NY, USA, 2010. Association for Computing Machinery. doi:10.1145/1835804.1835868.
- [18] Adrià Gascón, Yuval Ishai, Mahimna Kelkar, Baiyu Li, Yiping Ma, and Mariana Raykova. Computationally secure aggregation and private information retrieval in the shuffle model. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, page 4122–4136, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3670391.
- [19] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious rams. *J. ACM*, 43(3):431–473, may 1996. doi:10.1145/233551.233553.

- [20] Adam Groce, Peter Rindal, and Mike Rosulek. Cheaper private set intersection via differentially private leakage. *Proceedings on Privacy Enhancing Technologies*, 2019(3), 2019.
- [21] Paul Grubbs, Anurag Khandelwal, Marie-Sarah Lacharité, Lloyd Brown, Lucy Li, Rachit Agarwal, and Thomas Ristenpart. Pancake: Frequency smoothing for encrypted data stores. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2451–2468. USENIX Association, August 2020. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/grubbs>.
- [22] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with tiptoe. Cryptology ePrint Archive, Paper 2023/1438, 2023. URL: <https://eprint.iacr.org/2023/1438>, doi:10.1145/3600006.3613134.
- [23] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast Single-Server private information retrieval. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3889–3905, Anaheim, CA, August 2023. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/henzinger>.
- [24] Zhengli Huang and Wenliang Du. Optrr: Optimizing randomized response schemes for privacy-preserving data mining. In *2008 IEEE 24th International Conference on Data Engineering*, pages 705–714, 2008. doi:10.1109/ICDE.2008.4497479.
- [25] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Cryptography from anonymity. In *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*, pages 239–248, 2006. doi:10.1109/FOCS.2006.25.
- [26] Ishan Karunanayake, Nadeem Ahmed, Robert Malaney, Rafiqul Islam, and Sanjay K. Jha. De-anonymisation attacks on tor: A survey. *IEEE Communications Surveys; Tutorials*, 23(4):2324–2350, 2021. URL: <http://dx.doi.org/10.1109/COMST.2021.3093615>, doi:10.1109/comst.2021.3093615.
- [27] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz-Wu. Hintless single-server private information retrieval. Cryptology ePrint Archive, Paper 2023/1733, 2023. URL: <https://eprint.iacr.org/2023/1733>.
- [28] Xiaoyuan Liu, Ni Trieu, Trinabh Gupta, Ishtiyaque Ahmad, and Dawn Song. HADES: Range-filtered private aggregation on public data. Cryptology ePrint Archive, Paper 2024/1699, 2024. URL: <https://eprint.iacr.org/2024/1699>.
- [29] Wouter Lueks and Ian Goldberg. Sublinear scaling for multi-client private information retrieval. In Rainer Böhme and Tatsuaki Okamoto, editors, *Financial Cryptography and Data Security*, pages 168–186, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [30] Yiping Ma, Ke Zhong, Tal Rabin, and Sebastian Angel. Incremental Offline/Online PIR. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 1741–1758, Boston, MA, August 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/ma>.
- [31] Sonia Ben Mokhtar, Antoine Boutet, Pascal Felber, Marcelo Pasin, Rafael Pires, and Valerio Schiavoni. X-search: revisiting private web search using intel sgx. In *Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference*, Middleware '17, page 198–208, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3135974.3135987.
- [32] Muhammad Haris Mughees, Hao Chen, and Ling Ren. OnionPIR: Response efficient single-server PIR. Cryptology ePrint Archive, Paper 2021/1081, 2021. URL: <https://eprint.iacr.org/2021/1081>.
- [33] Hoang-Dung Nguyen, Jorge Guajardo, and Thang Hoang. Client-efficient online-offline private information retrieval. Cryptology ePrint Archive, Paper 2024/719, 2024. URL: <https://eprint.iacr.org/2024/719>.
- [34] Albin Petit, Thomas Cerqueus, Antoine Boutet, Sonia Mokhtar, David Coquil, Lionel Brunie, and Harald Kosch. Simattack: private web search under fire. *Journal of Internet Services and Applications*, 7, 12 2016. doi:10.1186/s13174-016-0044-x.
- [35] M.G. Reed, P.F. Syverson, and D.M. Goldschlag. Anonymous connections and onion routing. *IEEE Journal on Selected Areas in Communications*, 16(4):482–494, 1998. doi:10.1109/49.668972.
- [36] Hua Sun and Syed Ali Jafar. The capacity of private information retrieval. *IEEE Transactions on Information Theory*, 63(7):4075–4088, 2017. doi:10.1109/TIT.2017.2689028.
- [37] Raphael Toledo, George Danezis, and Ian Goldberg. Lower-cost epsilon-private information retrieval. *Proceedings on Privacy Enhancing Technologies*, 2016, 04 2016. doi:10.1515/popets-2016-0035.

- [38] Salil Vadhan. *The Complexity of Differential Privacy*, pages 347–450. Springer, Yehuda Lindell, ed., 2017. URL: https://link.springer.com/chapter/10.1007/978-3-319-57048-8_7.
- [39] Jelle van den Hooff, David Lazar, Matei Zaharia, and Nickolai Zeldovich. Vuvuzela: scalable private messaging resistant to traffic analysis. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP '15*, page 137–152, New York, NY, USA, 2015. Association for Computing Machinery. doi:10.1145/2815400.2815417.
- [40] David Wolinsky, Henry Corrigan-Gibbs, Bryan Ford, and Aaron Johnson. Scalable anonymous group communication in the anytrust model. *Fifth European Workshop on Systems Security*, 05 2012.
- [41] Xun Yi, Mohammed Golam Kaosar, Russell Paulet, and Elisa Bertino. Single-database private information retrieval from fully homomorphic encryption. *IEEE Transactions on Knowledge and Data Engineering*, 25(5):1125–1134, 2013. doi:10.1109/TKDE.2012.90.
- [42] Xiang Yue, Minxin Du, Tianhao Wang, Yaliang Li, Huan Sun, and Sherman S. M. Chow. Differential privacy for text analytics via natural text sanitization, 2021. URL: <https://arxiv.org/abs/2106.01221>, arXiv:2106.01221.
- [43] Wenyuan Zhao, Yu-Shin Huang, Ruida Zhou, and Chao Tian. Weakly private information retrieval from heterogeneously trusted servers. *IEEE Transactions on Information Theory*, 71(9):7292–7309, 2025. doi:10.1109/TIT.2025.3581850.
- [44] Mingxun Zhou, Andrew Park, Elaine Shi, and Wenting Zheng. Piano: Extremely simple, single-server PIR with sublinear server computation. *Cryptology ePrint Archive*, Paper 2023/452, 2023. URL: <https://eprint.iacr.org/2023/452>.

Appendix A Estimating δ and ϵ

The closed-form solution that calculates δ and ϵ includes combinatorics with very large parameters and probabilities with very low values, so calculating the security parameters for realistic scenarios, where $N \geq 10^5$ and $w \in O(N)$, is very difficult due to integer size limits and floating-point imprecision. Fortunately, the random variables X_i and X_j can be well-approximated by simple probability distributions, so Zeal can accurately estimate the needed w for required (ϵ, δ) values.

Specifically, each dummy is generated independently and the probability of a dummy being generated for any given index is $1/N$. The two random variables we are interested in, X_i and X_j , are not independent because a dummy being generated for i means it is not generated for j , but when N and w are on the order of millions, the dependence between them becomes incredibly small. Therefore,

$$X_i \approx \text{Bin}(w, \frac{1}{N}), X_j \approx \text{Bin}(w, \frac{1}{N}) \quad (18)$$

Where *Bin* refers to the binomial distribution.

For any required (ϵ, δ) values, $w \in O(N)$. In Zeal, w is about $10N$. Thus, $w * \frac{1}{N} \approx 10$. Since $1 \gg \frac{1}{N} > 0$, $w \gg 1$, and their product is moderate, this binomial distribution is well-approximated by a Poisson distribution, where $\lambda = w * \frac{1}{N} = \frac{w}{N}$. So

$$X_i \approx \text{Pois}(\frac{w}{N}), X_j \approx \text{Pois}(\frac{w}{N}), \quad (19)$$

With this model,

$$P[X_j = k] = e^{-\lambda} \lambda^k / k! \quad (20)$$

And

$$P[X_i \geq k] = 1 - \sum_{m=0}^{k-1} e^{-\lambda} \lambda^m / m! \quad (21)$$

Thus, we can quickly compute equation 14 by approximating each probability with this model. To accommodate the large factorials, we can use the logarithm of the Gamma function, and then take the $\exp()$ of the result, to preserve numerical stability. We provide a short Python script for this computation as a supplemental material.

Appendix B Batching

There is one significant optimization we can make to Zeal to eliminate unnecessary computations. The aforementioned tool Zeal uses to fool the adversary is sending dummy queries to *DB*. However, somewhat surprisingly, this is not what Zeal spends most of its time doing. Rather, most time is spent on the Diffie-Hellman key agreement algorithm. Naturally, to optimize Zeal, we should reduce the time spent during this step, maintaining the number of dummy requests while reducing latency. We call the following optimization *batching*.

Recall the above onion-like message structure. Only one index is combined with many keys and must be repeatedly decrypted. This makes sense when considering that each user should generate a unique key set, but we also introduce thousands of dummy requests that belong to no single user. With this in mind, we can group multiple indices so that they share cryptographic keys, reducing the number of expensive public-key operations each proxy performs. If we divide groups such that no two users' real requests are grouped together, every user may still generate a unique key set.

With batching, we form a 'batch' of B indices that all correspond to the same set of keys. This way, one Diffie-Hellman operation removes a layer from many messages. In total, the same number of dummies and real requests are needed, but they will be grouped.

For example, suppose there are 3 proxies, 1 real message, and meaningful security requires 3 dummies are sent. The initial request packet generated by the first proxy can be represented as a 2D array, with each row being one request (an index plus its unique keys):

$UPub_{11}$	$UPub_{12}$	$UPub_{13}$	User's encIndex
$UPub_{21}$	$UPub_{22}$	$UPub_{23}$	Dummy encIndex
$UPub_{31}$	$UPub_{32}$	$UPub_{33}$	Dummy encIndex
$UPub_{41}$	$UPub_{42}$	$UPub_{43}$	Dummy encIndex

Figure 9: Non-Batched packet structure, $y = 3$, $U = 1$

Each proxy must perform four Diffie-Hellman operations to compute their entire column of keys. Suppose we use $B = 3$ to batch this packet into groups and reduce latency:

$UPub_{11}$	$UPub_{12}$	$UPub_{13}$	User's index	Padding index 2	Padding index 3
$UPub_{21}$	$UPub_{22}$	$UPub_{23}$	Dummy index 1	Dummy index 2	Dummy index 3

Figure 10: Batched packet structure, $y = 3$, $U = 1$, $B = 3$. All indices are still encrypted but 'enc' is removed for brevity.

First, note that the number of Diffie-Hellman operations has been halved; each proxy performs only two, even though the total number of requests has increased.

Next, note that two 'padding indices' have been appended to the user's real request. These are generated exactly the same as dummy requests and are still dummies, but we give them this special name because they do not contribute to Zeal's security guarantee or fool the database. Rather, we need them so that the adversary cannot trivially distinguish real requests from dummies by observing that real requests are not grouped with others. P_1 generates padding indices itself as random bits, so the user does not have to download, create, or encrypt the extra data.

The reason why padding indices do not contribute to security is captured by our threat model (§2.1), which asserts that users other than the target user are assumed to be untrustworthy as the adversary may be monitoring or controlling their behavior. This means that the adversary likely knows the encryption keys associated with padding indices. If so, their contents will be revealed, so the adversary can strategically ignore them. This step does necessitate extra network communication cost, which we describe in the next section.

Zeal’s process must be slightly adjusted to accommodate batching: We must shuffle not only the order of batches but also the order of indices within each individual batch. This is because real requests always begin as the first index in a batch and must be mixed. Shuffling is already extremely fast compared to other operations and, since wider batches mean fewer of them in total, not much more shuffling must be done.

Security is not impacted by batching. Adversaries will gain the knowledge that at most one index within a batch could possibly be real, but they have no idea which one that may be. If both of the target user’s possible indices (i, j) appear, either as part of the same batch or within different batches, the user is protected.

Appendix B.1 Batching and Security

When we optimize Zeal with batching, we have to pad real requests with $B - 1$ random indices each. Note that B may be chosen as any integer, so it can be fine-tuned to maximize performance. As was mentioned before, these padding indices do not afford meaningful privacy to any user, because we assume adversaries have access to their corresponding keys and can unveil their contents. This means that, in batching, the total number of dummies that must be generated, y , is

$$y = w + (B - 1)(U) \quad (22)$$

where $(B - 1)(U)$ dummies are padding indices and may not contribute to the security guarantee. For a given database size and required level of security, Zeal determines the required value of w , but since y is not relevant to security and only affects performance, it will raise and lower as the load on Zeal increases and decreases. Fortunately, the $(B - 1)(U)$ term in y is not a bottleneck for large database sizes, as the number of dummies needed for any database with size N already exceeds N . When the database is small, and the $(B - 1)(U)$ term becomes more significant, latency is already naturally low, so we use a relatively large batching size of 50 in our experiments to reduce latency in situations where batching is most needed (§8.2). Overall, the padding indices are always dwarfed by the other dummies.

Appendix B.2 Batching in the Implementation

For our implementation and experiments, we set the batch size at $B = 50$ because, at this point, decrypting requests is

no longer Zeal’s bottleneck and sending large packets over the network is the largest source of latency.

Appendix C Pipelined Rounds

For our evaluation, we set the waiting period to 5 seconds for the queue. After 5 seconds, P_1 checks whether any requests have arrived since the last tick. If so, P_1 atomically snapshots the current queue and clears the live queue, so newly arriving requests are deferred to the next round. The snapshot is serialized and processed by a child goroutine as an independent round, which adds the necessary dummy batches and begins the forward pass (§2). If the forward slot is still occupied when a tick occurs, then P_1 does not start a new round until a later tick, so rounds are still defined by 5 second windows, but they only begin when P_1 is ready to forward them.

This design allows multiple rounds to be in flight across the proxy chain. Proxies P_2 through P_{final} maintain per-round state keyed by the RoundID, so chunks from different rounds can be received concurrently and buffered until all chunks for a given round have arrived. At the same time, each proxy limits its active computation with separate mutexes: it can perform at most one forward computation and at most one backward computation at a time. Thus, a proxy can pipeline two rounds concurrently, with one round moving forward while another completes the backward pass. Across a chain of P proxies, this yields at most $2P$ proxy computation stages active at once, plus up to one database stage when a round is being serviced by the database.

Let P denote the number of proxies and let μ denote the average time for a proxy to process one round on a single pass (forward or backward). The best-case service time for a request that is immediately admitted into a round is

$$T_{\text{best}} \approx 2P\mu + T_{\text{DB}}. \quad (23)$$

If a tick is skipped because P_1 ’s forward slot is still occupied, then admission can be delayed by roughly the remaining forward time on P_1 , giving the looser bound

$$T_{\text{worst}} \lesssim 2P\mu + T_{\text{DB}} + T_{\text{tick}} + \mu_{P_1, \text{fwd}}. \quad (24)$$